

Localizzazione Probabilistica e Stima dello Stato nella Robotica Mobile: Dai Filtri Bayesiani ai Modelli di Moto

NECKER

10 Giugno 2026

Indice

1	Richiami di Probabilità e Teorema di Bayes	2
1.1	Definizioni Fondamentali	2
1.2	Il Teorema di Bayes	2
2	Il Filtro di Bayes per la Stima dello Stato	2
2.1	L'Assunzione di Markov (Complete-State Assumption)	3
2.2	Algoritmo del Filtro di Bayes	3
2.3	Derivazione Formale del Filtro di Bayes	5
3	Implementazioni Pratiche del Filtro di Bayes	6
3.1	Il Filtro di Kalman (KF)	6
3.2	Extended Kalman Filter (EKF)	7
3.3	Filtri a Particelle (Monte Carlo Localization - MCL)	8
4	Action Models	9
4.1	Richiamo: Cinematica del Differential Drive	9
4.2	Modello Odometrico	10
4.3	Modello di Velocità	12
4.3.1	Il Limite Geometrico e l'Heading Drift	12
4.3.2	Valutazione della Probabilità $p(s_t s_{t-1}, u_t)$	12
4.3.3	Campionamento nel Filtro a Particelle (Sampling)	13
5	Modelli di Percezione	14
5.1	Beam-Based Model (Modello basato sul raggio)	14
5.2	Likelihood Field Model (Scan Model)	16

1 Richiami di Probabilità e Teorema di Bayes

La probabilità è il fondamento del ragionamento in condizioni di incertezza, caratteristica intrinseca della robotica autonoma.

1.1 Definizioni Fondamentali

- **Probabilità di un evento A :** $0 \leq P(A) \leq 1$.
- **Probabilità Congiunta:** $P(A, B)$ rappresenta la probabilità che A e B si verifichino simultaneamente. Se A e B sono indipendenti, allora $P(A, B) = P(A)P(B)$.
- **Legge della Probabilità Totale:** Per una variabile discreta X , la probabilità marginale è $P(X) = \sum_y P(X|Y)P(Y)$. Nel caso continuo, diventa un integrale: $p(x) = \int p(x|y)p(y)dy$.

1.2 Il Teorema di Bayes

Il Teorema di Bayes fornisce uno strumento per aggiornare le nostre credenze (probabilità a posteriori) basandoci su nuove evidenze (osservazioni sensoriali). Matematicamente:

$$P(A|E) = \frac{P(E|A)P(A)}{P(E)} \quad (1)$$

Dove:

- $P(A|E)$ è la **probabilità a posteriori** (diagnostica): qual è la probabilità della causa A dato l'effetto osservato E ?
- $P(E|A)$ è la **Verosimiglianza (Likelihood)** (causale): se A fosse vero, quanto sarebbe probabile osservare l'effetto E ?
- $P(A)$ è la **Probabilità a priori (Prior)**: la nostra convinzione iniziale su A prima di osservare l'evidenza.
- $P(E)$ è l'**Evidenza**: agisce da fattore di normalizzazione.

2 Il Filtro di Bayes per la Stima dello Stato

In robotica autonoma, lo stato reale di un sistema (ad esempio, la posa globale del robot $x_t = \langle x, y, \theta \rangle$) non è mai direttamente e perfettamente accessibile. Sensori e attuatori sono soggetti a rumore termico, sistematico e ambientale. Pertanto, il robot non può sapere con assoluta certezza dove si trova, ma deve stimarlo basandosi sullo storico delle azioni di controllo $u_{1:t}$ (es. velocità comandate ai motori) e sulle misurazioni dei sensori $z_{1:t}$ (es. letture LiDAR o odometria).

Per gestire questa incertezza, il robot mantiene e aggiorna continuamente una "credenza" (*belief*) sullo stato, rappresentata da una distribuzione di probabilità condizionata:

$$Bel(x_t) = p(x_t | z_{1:t}, u_{1:t}) \quad (2)$$

Questa espressione indica la probabilità che il robot si trovi nello stato x_t , data tutta l'informazione acquisita dal tempo 1 fino al tempo corrente t .

2.1 L'Assunzione di Markov (Complete-State Assumption)

Calcolare direttamente $Bel(x_t)$ tenendo conto di un intero storico che cresce linearmente nel tempo (da $t = 1$ fino a t) comporterebbe un onere computazionale ingestibile ($O(t)$ ad ogni iterazione) e richiederebbe una memoria infinita.

Per rendere il problema trattabile e ricorsivo, si introduce la fondamentale **Assunzione di Markov**. Essa stabilisce che lo stato presente x_t è un riassunto *completo e sufficiente* di tutta la storia passata. Se conosciamo lo stato attuale x_t , tutto ciò che è accaduto prima non fornisce alcuna informazione aggiuntiva utile per prevedere il futuro.

ATTENZIONE: l'assunzione di markov viene applicata su tutte le possibili posizioni del robot calcolate nella belief, questa assunzione serve solo per ridurre il numero di informazioni necessarie per calcolare lo stato futuro

Questa assunzione è composta da due fondamentali indipendenze condizionali:

- **Indipendenza delle misurazioni (Modello di Percezione):** La misurazione attuale z_t dipende *esclusivamente* dallo stato attuale x_t . Conoscere le letture passate dei sensori non aiuta a prevedere la lettura attuale se si conosce già l'esatta posizione del robot:

$$p(z_t | x_{0:t}, z_{1:t-1}, u_{1:t}) = p(z_t | x_t) \quad (3)$$

- **Indipendenza delle transizioni di stato (Modello di Moto):** L'evoluzione verso lo stato attuale x_t dipende *esclusivamente* dallo stato immediatamente precedente x_{t-1} e dall'azione di controllo u_t appena eseguita. Il percorso specifico fatto per arrivare a x_{t-1} è irrilevante:

$$p(x_t | x_{0:t-1}, z_{1:t}, u_{1:t}) = p(x_t | x_{t-1}, u_t) \quad (4)$$

Queste relazioni di dipendenza e indipendenza possono essere visualizzate tramite il seguente grafo (Modello di Markov Nascosto - HMM):

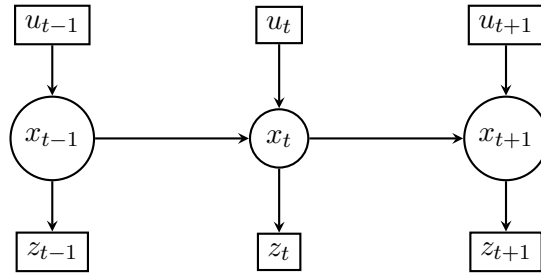


Figura 1: Modello di Markov Nascosto (HMM) applicato alla robotica mobile. Gli stati x sono "nascosti", mentre le misurazioni z e le azioni u sono osservabili.

2.2 Algoritmo del Filtro di Bayes

Il Punto di Partenza (Tempo $t = 0$):

Prima che il robot compia qualsiasi azione, deve avere una condizione iniziale nella sua memoria RAM. Questa è la credenza iniziale, o prior assoluto: $Bel(x_0)$.

- Se il robot sa esattamente dove si trova all'accensione (es. coordinate di partenza fisse), $Bel(x_0)$ sarà un singolo picco (certezza del 100% su un punto).
- Se il robot viene acceso "alla cieca" in un punto sconosciuto (Global Localization), $Bel(x_0)$ sarà una curva piatta: tutte le posizioni nello spazio hanno la stessa identica probabilità.

Il Ciclo Ricorsivo (Da $t - 1$ a t):

A questo punto, il ciclo inizia. Supponiamo di trovarci in un istante generico t . Nella memoria del robot è rimasta salvata **esclusivamente** la curva di probabilità finale calcolata al passo precedente: $Bel(x_{t-1})$. Tutto il resto della storia passata è stato cancellato.

Il robot esegue due passaggi sequenziali per costruire la nuova curva $Bel(x_t)$:

1. Fase 1: Predizione (Spingere la curva in avanti usando Markov)

Il robot ha appena eseguito un comando di movimento u_t (es. "avanza 1 metro"). Il suo obiettivo ora è creare una nuova curva temporanea, chiamata *Belief Predetto* ($\overline{Bel}(x_t)$).

Come la crea? Sfrutta l'Assunzione di Markov applicata alla **Legge della Probabilità Totale**:

$$\overline{Bel}(x_t) = \int p(x_t|u_t, x_{t-1})Bel(x_{t-1})dx_{t-1} \quad (5)$$

- $p(x_t|u_t, x_{t-1})$ – Rappresenta la probabilità condizionata di raggiungere esattamente lo stato x_t , dato che il robot è partito dal punto specifico x_{t-1} e ha eseguito il comando u_t . Questo termine è il diretto responsabile della "spalmatura" dell'incertezza dovuta a fattori come lo slittamento delle ruote o la deriva dell'odometria.
- $Bel(x_{t-1})$ – È la curva di probabilità consolidata al termine del ciclo precedente.

Il risultato fisico è che l'intera vecchia curva viene letteralmente "traslata in avanti" nello spazio e contemporaneamente "spalmata" (allargata), poiché il movimento fisico introduce sempre nuova incertezza (es. slittamento delle ruote).

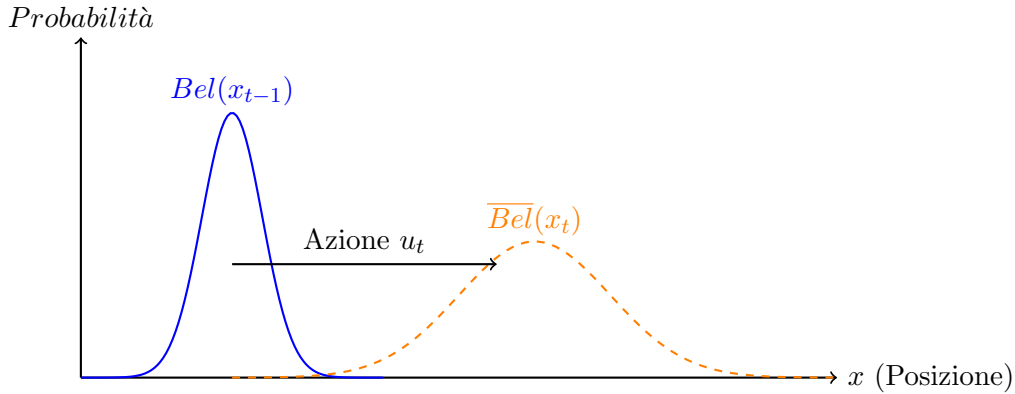


Figura 2: **Predizione:** Il robot si muove in avanti. La curva viene traslata verso la nuova posizione, ma si abbassa e si allarga (aumento della varianza) a causa dell'incertezza intrinseca del movimento.

Importante: Appena la nuova curva predetta $\overline{Bel}(x_t)$ è stata disegnata, la vecchia $Bel(x_{t-1})$ viene definitivamente eliminata dalla memoria.

2. Fase 2: Correzione (Tagliare la curva usando i Sensori)

La curva $\overline{Bel}(x_t)$ appena creata è però troppo incerta (spalmata). Il robot acquisisce in questo esatto momento una nuova lettura dai sensori z_t (es. il LiDAR rileva un muro a 20 cm).

Il robot ha come obiettivo, usare la curva di probabilità di essere in quella pos in base alla rilevazione del sensore (se mi trovo a $x = 60m$ e il muro è stato individuato a $2.5m$ e la pos del muro è $x = 70$ allora in quel punto la probabilità sarà molto bassa), per modellare la prior. Applica il Teorema di Bayes moltiplicando la curva predetta per il **Modello di**

Percezione (il risultato è una sorta di interpolazione tra le due curve):

$$Bel(x_t) = \eta \cdot p(z_t|x_t)\overline{Bel}(x_t) \quad (6)$$

Il robot si pone una domanda per ogni punto della curva predetta: *"Se mi trovassi fisicamente in questo specifico punto, quanto sarebbe probabile vedere un muro a 20 cm?"*.

- Se la risposta è "molto probabile", la curva in quel punto viene mantenuta alta o alzata.
- Se la risposta è "impossibile" (es. in quel punto sulla mappa non ci sono muri vicini), la curva in quel punto viene schiacciata a zero.

L'operazione restringe drasticamente la curva di probabilità attorno alle posizioni che hanno senso logico rispetto alla lettura del sensore. Il parametro η agisce come semplice **fattore di normalizzazione** matematico: riscalda tutta la curva verso l'alto o verso il basso affinché l'area totale sottesa torni a valere esattamente 1 (condizione obbligatoria per una distribuzione di probabilità).

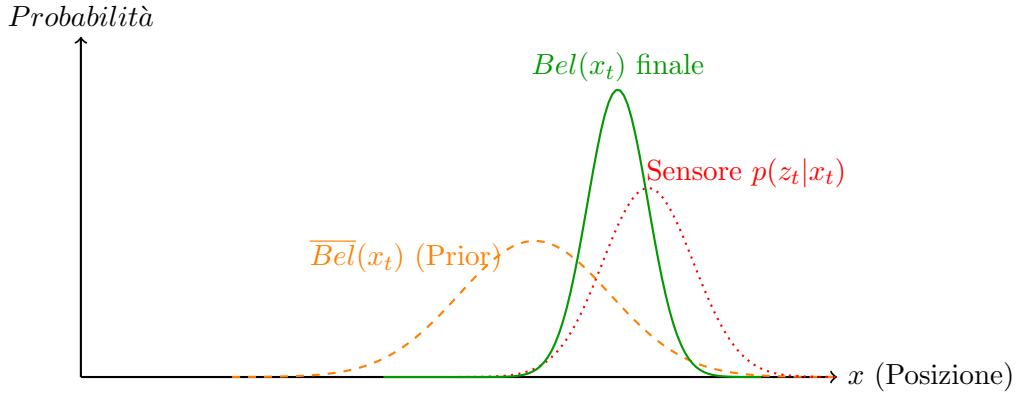


Figura 3: **Correzione (Moltiplicazione di Bayes)**: La curva predetta (arancione) viene fusa con la curva del sensore (rossa). Il risultato (verde) è un compromesso tra le due: l'incertezza si riduce drasticamente e la probabilità si concentra attorno alla vera posizione del robot.

Al termine della Fase 2, la curva $\overline{Bel}(x_t)$ viene cancellata. In memoria rimane solo il nuovo *Belief* finale $Bel(x_t)$, e il robot è pronto ad aspettare il prossimo comando per ricominciare il ciclo identico al tempo $t + 1$.

NOTARE CHE NEL MODELLO TEORICO NON SONO TUTTE CURVE GAUSSIANE (generalmente rappresentano le prob di trovarci in una determinata posizione, ma se per esempio abbiamo 3 porte uguali avremo tre picchi e non uno come nella gaussiana), **!GUARDARE FILTRO DI KALMAN!**

2.3 Derivazione Formale del Filtro di Bayes

La spiegazione concettuale appena vista (Predizione e Correzione) può essere derivata in modo rigoroso partendo dalla definizione della *belief* come probabilità congiunta condizionata. L'obiettivo è calcolare e semplificare l'espressione $Bel(x_t) = p(x_t|u_{1:t}, z_{1:t})$.

Procediamo applicando le leggi fondamentali della probabilità:

1. **Applicazione del Teorema di Bayes:** Trattiamo l'ultima misurazione z_t come evidenza e tutto lo storico precedente come conoscenza pregressa. Il denominatore $P(E)$ viene assorbito in un fattore di normalizzazione η :

$$Bel(x_t) = \eta \cdot p(z_t|x_t, u_{1:t}, z_{1:t-1}) \cdot p(x_t|u_{1:t}, z_{1:t-1})$$

2. **Assunzione di Markov (Modello di Percezione):** Poiché la misura z_t dipende unicamente dallo stato attuale x_t e non da ciò che è accaduto in passato, il primo termine si semplifica:

$$Bel(x_t) = \eta \cdot p(z_t|x_t) \cdot p(x_t|u_{1:t}, z_{1:t-1})$$

3. **Legge della Probabilità Totale:** Per collegare l'istante attuale a quello precedente, introduciamo lo stato x_{t-1} integrandolo su tutto il suo dominio all'interno del secondo termine:

$$Bel(x_t) = \eta \cdot p(z_t|x_t) \int p(x_t|x_{t-1}, u_{1:t}, z_{1:t-1}) \cdot p(x_{t-1}|u_{1:t}, z_{1:t-1}) dx_{t-1}$$

4. **Assunzione di Markov (Modello di Moto) e Ricorsione:** Sappiamo che l'evoluzione nello stato x_t dipende solo da x_{t-1} e dall'ultima azione u_t . Inoltre, notiamo che l'ultimo termine dell'integrale $p(x_{t-1}|u_{1:t-1}, z_{1:t-1})$ non è altro che la definizione esatta della *belief* al passo precedente, $Bel(x_{t-1})$. Sostituendo otteniamo:

$$Bel(x_t) = \eta \cdot p(z_t|x_t) \int p(x_t|x_{t-1}, u_t) \cdot Bel(x_{t-1}) dx_{t-1}$$

Abbiamo così ottenuto la formula ricorsiva finale del Filtro di Bayes, che divide in modo netto l'aggiornamento dovuto all'azione (l'integrale di predizione) dall'aggiornamento dovuto all'osservazione (la moltiplicazione correttiva esterna).

3 Implementazioni Pratiche del Filtro di Bayes

Il Filtro di Bayes visto nella sua formulazione teorica è continuo e richiede di risolvere integrali complessi su spazi di stato multidimensionali. Su un computer reale (che ha memoria e potenza di calcolo limitate), questo calcolo esatto è impossibile.

Per questo motivo, la robotica utilizza delle implementazioni approssimate, che si dividono in due grandi famiglie:

- **Filtri Parametrici** (come il Filtro di Kalman) che assumono distribuzioni fisse (Gaussiane).
- **Filtri Non-Parametrici** (come i Filtri a Particelle) che usano campionamenti discreti.

3.1 Il Filtro di Kalman (KF)

Il Filtro di Kalman è la soluzione analitica perfetta al Filtro di Bayes, valida solo sotto tre fortissime assunzioni:

1. La *belief* iniziale è una distribuzione Gaussiana. (se il robot fosse in una situazione di localizzazione globale con 3 picchi distinti per 3 porte identiche), il filtro fallirebbe all'istante $t=0$. Non saprebbe come descrivere quei 3 picchi usando solo una media e una covarianza, poiché una Gaussiana per definizione ha un solo e unico picco (è unimodale).
2. Il modello di moto e il modello di percezione sono **strettamente lineari**.
3. Il rumore aggiunto dal moto (R_t) e dai sensori (Q_t) è puramente Gaussiano e a media nulla.

Se queste condizioni sono rispettate, la probabilità rimane per sempre una Gaussiana esatta. Di conseguenza, il robot non ha bisogno di memorizzare una curva complessa, ma gli bastano solo due parametri (da cui il nome filtro parametrico):

- **Il vettore media μ :** La stima della posizione più probabile.
- **La matrice di covarianza Σ :** L'incertezza associata alla stima.

Le equazioni del Filtro di Kalman trasformano gli integrali di Bayes in semplici e velocissime moltiplicazioni tra matrici:

$$\textbf{Predizione (Moto): } \bar{\mu}_t = A_t \mu_{t-1} + B_t u_t \quad (7)$$

$$\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t \quad (8)$$

$$\textbf{Correzione (Sensori): } K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1} \quad (\text{Guadagno di Kalman}) \quad (9)$$

$$\mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t) \quad (10)$$

$$\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t \quad (11)$$

3.2 Extended Kalman Filter (EKF)

Il limite del Filtro di Kalman puro è che i robot reali **non sono lineari**. Come abbiamo visto nella cinematica, per calcolare il moto compaiono seni e coseni (se il robot si sposta inclinato con un determinato angolo, si sposterà di $\cos(\alpha)$ sulle x e di $\sin(\alpha)$ sulle y), e i sensori misurano distanze euclidee (radici quadrate e arcotangenti). Se passiamo una Gaussiana attraverso una funzione non lineare (es. il modello di velocità $g(x)$, cioè l'equazione reale del moto), la curva si deforma a "banana" e non è più una Gaussiana. Questo distruggerebbe la matematica del KF.

L'**Extended Kalman Filter (EKF)** risolve il problema usando l'approssimazione di Taylor al primo ordine. L'EKF *linearizza localmente* le funzioni non lineari g (moto) calcolando la sua derivate prime, ovvero la **Matrice Jacobiana G_t** , valutate esattamente nel punto medio μ della stima corrente.

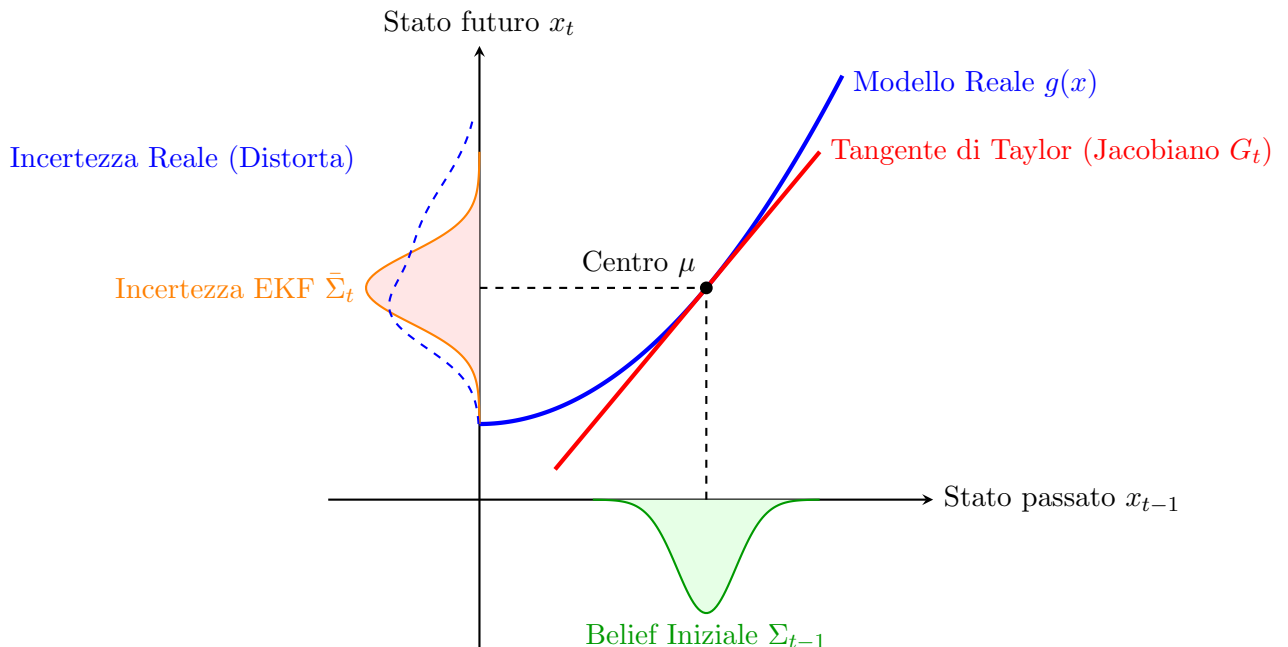


Figura 4: **La meccanica dell'EKF:** L'incertezza iniziale (verde) viene proiettata non attraverso la curva fisica (blu), ma attraverso la retta tangente di Taylor (rossa). Questo trucco forza il risultato (arancione) a rimanere una Gaussiana perfetta, sebbene si discosti leggermente dalla realtà asimmetrica (curva tratteggiata blu).

Come si nota dal grafico, proiettando la gaussiana iniziale sulla retta di incertezza rossa, si ottiene sull'asse y una gaussiana perfetta (più allungata) essa avrà la media spostata (intersezione tra vecchia media e retta), infine questa nuova gaussiana sarà riportata sull'asse x .

ATTENZIONE: tutto questo processo si sostituisce alla primissima parte dell'algoritmo del filtro di Bayes nel caso usiamo funzioni non lineari

3.3 Filtri a Particelle (Monte Carlo Localization - MCL)

I filtri Gaussiani (KF e EKF) gestiscono distribuzioni unimodali (il robot crede di essere in un solo posto). Se un robot viene acceso alla cieca (*Global Localization*) o rapito e spostato (*Kidnapped Robot Problem*), l'EKF fallisce irrimediabilmente.

Il **Filtro a Particelle** è un approccio non-parametrico che rappresenta il *Belief* non con formule matematiche, ma con uno sciame discreto di migliaia di *particelle*. Ogni particella $x_t^{[m]}$ rappresenta una singola e completa ipotesi sullo stato del robot.

L'algoritmo esegue iterativamente tre fasi magiche ispirate alla selezione naturale:

1. **Campionamento / Predizione (Sampling):** Il robot acquisisce il comando odometrico u_t . Per *ogni singola particella*, si applica il Modello di Moto (es. Modello Odometrico), calcolando lo spostamento teorico e sommandovi un errore Gaussiano casuale. Le particelle si spostano e la "nuvola" si disperde.
2. **Assegnazione dei Pesi / Correzione (Importance Weighting):** Il robot acquisisce la lettura del laser z_t . Interrogando il Modello di Percezione (es. Likelihood Field), si calcola per ogni particella quanto è verosimile quella lettura se il robot si trovasse esattamente lì. Questa probabilità diventa il **peso di importanza** $w_t^{[m]}$ della particella.
3. **Ricampionamento (Resampling):** È il cuore dell'algoritmo. Si crea un nuovo set di particelle partendo da zero. Si estrae "alla cieca" dal set precedente usando una ruota della fortuna truccata (dove gli spicchi sono proporzionali ai pesi w).
 - Le particelle con pesi alti (che combaciano bene coi sensori) vengono estratte più volte (si *clonano*).
 - Le particelle con pesi bassi (fuori rotta o dentro un muro) vengono ignorate (si *estinguono*).

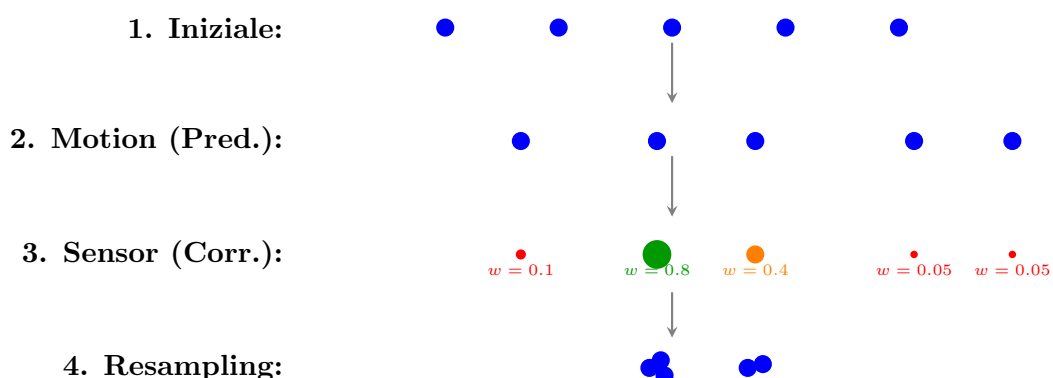


Figura 5: Ciclo vitale del Filtro a Particelle. Le misurazioni dei sensori non spostano le particelle, ma ne alterano il "peso". Il resampling elimina le ipotesi errate e concentra la potenza di calcolo dove il robot pensa di essere.

4 Action Models

I modelli di moto descrivono l'evoluzione probabilistica dello stato del robot. Il loro scopo è calcolare $p(s_t | s_{t-1}, u_t)$, ovvero la probabilità che il robot si trovi nella posa s_t , sapendo che era in s_{t-1} e ha eseguito l'azione u_t . In implementazioni reali (come EKF o Particle Filter), questi modelli vengono usati per due task principali:

- **Evaluation (Valutazione):** Dati s_{t-1} , u_t e un'ipotesi di arrivo s_t , calcolare quanto è probabile questa transizione.
- **Sampling (Campionamento):** Dati s_{t-1} e u_t , generare casualmente una possibile posa di arrivo s_t aggiungendo il rumore corretto.

4.1 Richiamo: Cinematica del Differential Drive

Prima di introdurre i modelli probabilistici, è utile richiamare le equazioni cinematiche dirette di un robot a guida differenziale (il classico robot a due ruote motrici indipendenti, comunemente usato per lo *skid steering*). Questo ci aiuta a comprendere su quali leve fisiche agisce l'azione u_t .

In un ambiente planare 2D, definiamo i parametri costruttivi e di controllo del robot:

- R : raggio delle ruote.
- L : distanza tra le due ruote (interasse).
- ω_R, ω_L : velocità angolari rispettivamente della ruota destra e della ruota sinistra.

Se il robot si trova con un orientamento θ rispetto all'asse globale di riferimento, le equazioni di transizione che descrivono le velocità lineari e la velocità angolare del suo centro (la posa x, y, θ) sono date da:

$$\dot{x} = \frac{R}{2}(\omega_R + \omega_L) \cos(\theta) \quad (12)$$

$$\dot{y} = \frac{R}{2}(\omega_R + \omega_L) \sin(\theta) \quad (13)$$

$$\dot{\theta} = \frac{R}{L}(\omega_R - \omega_L) \quad (14)$$

In un sistema di controllo a tempo discreto con intervallo temporale Δt , la posa successiva si può approssimare integrando queste velocità:

$$x(t + \Delta t) \approx x(t) + \dot{x}\Delta t \quad (15)$$

$$y(t + \Delta t) \approx y(t) + \dot{y}\Delta t \quad (16)$$

$$\theta(t + \Delta t) \approx \theta(t) + \dot{\theta}\Delta t \quad (17)$$

Questo modello, basato esclusivamente sulla cinematica in avanti (*Forward Kinematics*), è puramente **deterministico**. Presuppone cioè condizioni ideali e un mondo perfetto. Tuttavia, se usato per la navigazione a lungo termine (*Dead Reckoning*), è destinato a fallire: lo slittamento delle ruote, le asimmetrie di attrito e le variazioni di diametro introducono errori non prevedibili.

Per ovviare a questo problema, i robot mobili non utilizzano la cinematica pura, ma vi sovrappongono una valutazione statistica del rumore, dando vita ai modelli probabilistici che esamineremo ora: il **Modello Odometrico** e il **Modello di Velocità**.

4.2 Modello Odometrico

Il modello odometrico viene utilizzato quando il robot dispone di sensori propriocettivi (misurano cosa effettivamente succede durante il movimento, come gli encoder montati sulle ruote di un *differential drive*). Gli encoder misurano la rotazione fisica delle ruote, permettendo di calcolare una stima (spesso affetta da deriva) del movimento compiuto.

Invece di lavorare direttamente con le velocità delle ruote, l'azione u_t viene codificata estraendo la differenza tra la posa odometrica letta all'istante $t - 1$ e quella letta all'istante t . Qualsiasi movimento su un piano 2D può essere scomposto in una sequenza geometricamente equivalente di tre fasi:

1. **Prima Rotazione** (δ_{rot1}): Il robot ruota sul posto per allinearsi alla direzione della destinazione.
2. **Traslazione** (d): Il robot avanza in linea retta verso la destinazione.
3. **Seconda Rotazione** (δ_{rot2}): Il robot ruota sul posto per assumere l'orientamento finale desiderato.

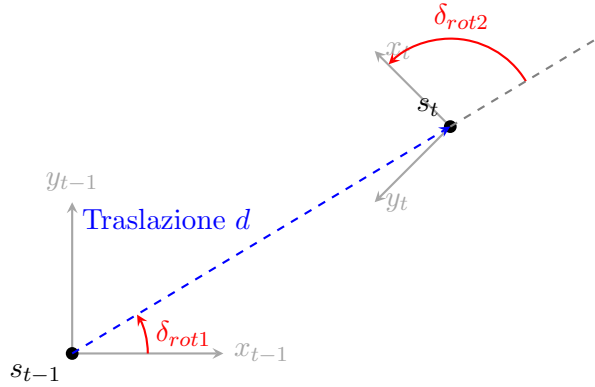


Figura 6: Scomposizione odometrica del movimento in tre fasi (Rotazione - Traslazione - Rotazione).

Valutazione della Probabilità nel Modello Odometrico:

Il modello non genera una nuova posa, ma risponde alla domanda: "Dato lo spostamento odometrico misurato dai sensori $u_t = \langle \hat{\delta}_{rot1}, \hat{d}, \hat{\delta}_{rot2} \rangle$, qual è la probabilità $p(s_t | s_{t-1}, u_t)$ che il robot si trovi effettivamente nella posa ipotetica s_t ?"

Per calcolarla, procediamo in due fasi:

1. **Recupero dei parametri cinematici** ($rot \rightarrow tras \rightarrow rot$):

Calcoliamo il movimento analitico esatto $\langle \delta_{rot1}, d, \delta_{rot2} \rangle$ che sarebbe stato necessario per passare matematicamente dalla posa precedente s_{t-1} alla posa ipotetica s_t . Utilizziamo questa scomposizione perché ci permette di separare la traslazione dalle rotazioni, potendo quindi calcolare gli errori su ciascuna componente in modo indipendente.

2. **Valutazione statistica degli errori:**

Confrontiamo il movimento teoricamente necessario $\langle \delta_{rot1}, d, \delta_{rot2} \rangle$ con il movimento effettivamente misurato dai sensori $\langle \hat{\delta}_{rot1}, \hat{d}, \hat{\delta}_{rot2} \rangle$. La differenza tra i due rappresenta l'errore Δ calcolato su ciascuna componente (es. $\Delta_d = d - \hat{d}$).

Valutiamo la probabilità di commettere questo errore usando una distribuzione Gaussiana $p_\sigma(\Delta)$ centrata sullo zero (che rappresenta la condizione ideale di errore nullo). Il funzionamento logico e visivo di questa valutazione si basa su due pilastri:

- **La forma della campana (Varianza σ^2):** Dipende dall'ampiezza del movimento misurato e dai parametri di rumore. Un movimento odometrico breve genera una campana stretta e alta (bassa incertezza); un movimento lungo genera una campana larga e schiacciata (alta incertezza).
- **Il prelievo del valore (L'errore Δ):** L'errore funge da coordinata x sul grafico. Entrando nella campana con il nostro Δ , leggiamo l'altezza corrispondente sull'asse y . Un Δ prossimo allo zero restituirà una probabilità elevata, mentre un Δ elevato ci farà cadere sulle "code" della campana, indicando una probabilità quasi nulla.
(che la pos indicata dall'odimetria sia quella vera)

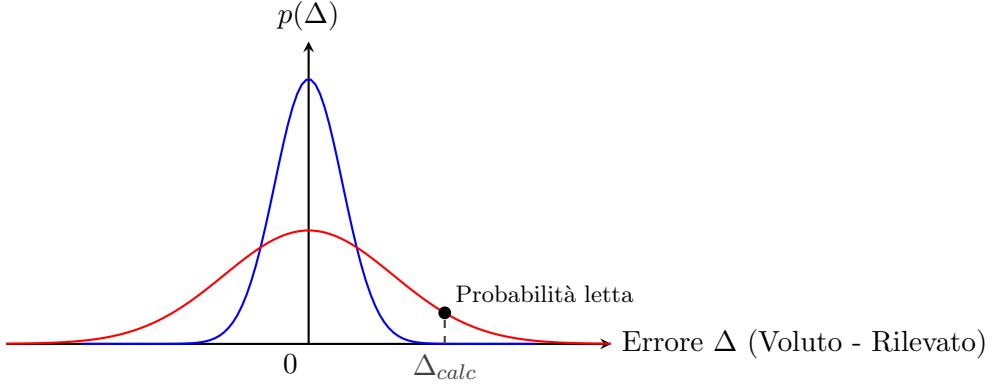


Figura 7: Valutazione della probabilità dell'errore. La larghezza della campana è definita dal movimento misurato e dai parametri α ; il Δ determina il punto di lettura.

Nello specifico, la varianza (che definisce quanto si "allarga" la campana) per ogni componente viene calcolata pesando le letture odometriche attraverso i parametri di rumore cinematico intrinseci del robot ($\alpha_1, \alpha_2, \alpha_3, \alpha_4$):

- **Errore di rotazione (σ_{rot1}^2 e σ_{rot2}^2):** L'incertezza aumenta drasticamente sia per ampie sterzate, sia durante lunghe traslazioni (es. slittamento asimmetrico delle ruote a terra). È quantificata come:

$$\sigma_{rot1}^2 = \alpha_1 \hat{\delta}_{rot1}^2 + \alpha_2 \hat{d}^2$$

$$\sigma_{rot2}^2 = \alpha_1 \hat{\delta}_{rot2}^2 + \alpha_2 \hat{d}^2$$

- **Errore di traslazione (σ_d^2):** L'incertezza cresce se il robot percorre lunghi tratti lineari (es. differenze millimetriche nel diametro delle ruote), ed è influenzata dalle rotazioni. È quantificata come:

$$\sigma_d^2 = \alpha_3 \hat{d}^2 + \alpha_4 (\hat{\delta}_{rot1}^2 + \hat{\delta}_{rot2}^2)$$

4.3 Modello di Velocità

A differenza del modello odometrico, che si basa sulle misurazioni dei sensori a posteriori, il **Modello di Velocità** si basa sui comandi di controllo inviati al robot. L'azione $u_t = \langle v, \omega \rangle$ specifica la velocità lineare v e la velocità angolare ω che il controller ordina di mantenere per un intervallo di tempo Δt .

4.3.1 Il Limite Geometrico e l'Heading Drift

Quando si inviano comandi di velocità costanti $\langle v, \omega \rangle$, la cinematica teorica stabilisce che il robot si muoverà percorrendo un **arco di cerchio perfetto**.

Se provassimo a valutare la probabilità $p(s_t|s_{t-1}, u_t)$ calcolando semplicemente l'arco di cerchio unico che connette la posizione di partenza (x, y) a quella di arrivo (x', y') , ci scontreremmo con un limite strutturale del modello: la geometria di un arco perfetto vincola in modo rigido anche l'orientamento finale. Il robot dovrebbe arrivare nella posa finale con un angolo θ' perfettamente tangente all'arco. Qualsiasi posa ipotetica s_t che presenti un angolo di arrivo non coincidente con la tangente riceverebbe matematicamente una probabilità pari a zero, rendendo il filtro inutilizzabile nel mondo reale (dove dossi e slittamenti modificano continuamente l'orientamento).

Per risolvere questo problema, il modello introduce un terzo parametro chiamato **Heading Drift** ($\hat{\gamma}$). Questo valore quantifica la rotazione extra imprevista, permettendo al modello di tollerare e valutare le pose in cui il robot è arrivato a destinazione, ma risulta "fuori asse" rispetto alla traiettoria ideale.

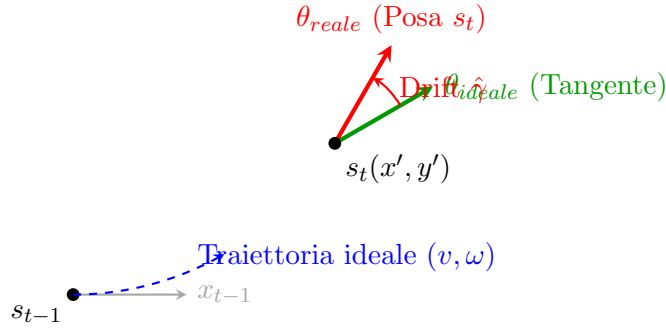


Figura 8: L'introduzione dell'Heading Drift ($\hat{\gamma}$) permette di giustificare matematicamente pose finali che non sono perfettamente tangenti all'arco di movimento teorico.

4.3.2 Valutazione della Probabilità $p(s_t|s_{t-1}, u_t)$

Durante l'aggiornamento del filtro, per calcolare la probabilità che il robot si trovi nella posa candidata s_t , il sistema opera nel seguente modo:

1. **Recupero delle velocità ideali:** Calcola analiticamente quali velocità $\langle \hat{v}, \hat{\omega} \rangle$ e quale drift finale $\hat{\gamma}$ sarebbero stati necessari per passare esattamente da s_{t-1} a s_t nel tempo Δt .
2. **Calcolo delle discrepanze:** Confronta le velocità comandate $\langle v, \omega \rangle$ con quelle idealmente necessarie $\langle \hat{v}, \hat{\omega} \rangle$, ottenendo gli errori.
3. **Valutazione statistica (Prodotto di Gaussiane):** Valuta i tre errori tramite tre distribuzioni Normali. La probabilità totale è il prodotto dei "voti" assegnati alle tre componenti:

$$p(s_t|s_{t-1}, u_t) = p_v(v - \hat{v}) \cdot p_\omega(\omega - \hat{\omega}) \cdot p_\gamma(\hat{\gamma})$$

Le varianze σ^2 che definiscono la larghezza delle tre campane di Gauss dipendono dai parametri cinematici di rumore del robot ($\alpha_1 \dots \alpha_6$) e crescono all'aumentare delle velocità comandate:

- **Varianza sulla velocità lineare:** $\sigma_v^2 = \alpha_1 v^2 + \alpha_2 \omega^2$
- **Varianza sulla velocità angolare:** $\sigma_\omega^2 = \alpha_3 v^2 + \alpha_4 \omega^2$
- **Varianza sul drift finale:** $\sigma_\gamma^2 = \alpha_5 v^2 + \alpha_6 \omega^2$

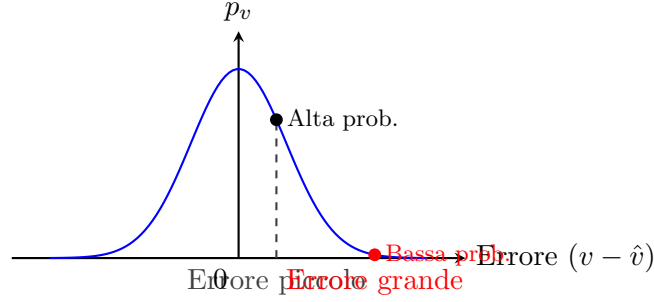


Figura 9: Esempio di valutazione per la componente di velocità lineare. Lo stesso principio a campana viene applicato alla velocità angolare e al drift $\hat{\gamma}$.

4.3.3 Campionamento nel Filtro a Particelle (Sampling)

Nel contesto della *Monte Carlo Localization*, il modello viene usato in senso generativo per "muovere" lo sciame di particelle. Partendo dalla posa s_{t-1} e ricevendo l'azione $u_t = \langle v, \omega \rangle$, si genera la nuova posa s_t applicando un rumore casuale estratto dalle rispettive distribuzioni gaussiane:

1. Si "sporca" il comando lineare: $\hat{v}_{noisy} = v + \text{Sample}(\mathcal{N}(0, \sigma_v^2))$
2. Si "sporca" il comando angolare: $\hat{\omega}_{noisy} = \omega + \text{Sample}(\mathcal{N}(0, \sigma_\omega^2))$
3. Si estrae uno scostamento angolare casuale: $\hat{\gamma}_{noisy} = \text{Sample}(\mathcal{N}(0, \sigma_\gamma^2))$

Infine, si propaga la particella per il tempo Δt utilizzando le equazioni di cinematica diretta con le velocità modificate $(\hat{v}_{noisy}, \hat{\omega}_{noisy})$ e si aggiunge l'angolo di drift $\hat{\gamma}_{noisy}$ all'orientamento finale, ottenendo così la nuova posa ipotetica.

5 Modelli di Percezione

Dopo aver stimato dove si trova in base al movimento (Predizione), il robot deve usare i propri sensori esocettivi (telecamere, LiDAR, sonar) per "guardare" l'ambiente e correggere la propria stima (Correzione). Il Modello di Percezione ha il compito di calcolare la probabilità dell'osservazione:

$$p(z_t|x_t, m) \quad (18)$$

Questa formula risponde alla domanda: *"Se il robot si trovasse esattamente nella posa ipotetica x_t all'interno della mappa m , quanto sarebbe probabile ottenere l'esatta misurazione z_t che i sensori hanno appena restituito?"*

L'ambiente reale viene solitamente fornito al robot sotto forma di **Occupancy Grid Map** (Mappa a Griglia di Occupazione), dove lo spazio è diviso in celle e ogni cella contiene la probabilità di essere occupata da un ostacolo.

Per sensori telemetrici come i laser scanner (LiDAR), che restituiscono un array di distanze (raggi), esistono due modelli matematici principali: il *Beam-based Model* e il *Likelihood Field Model*.

5.1 Beam-Based Model (Modello basato sul raggio)

Il modello Beam-based è molto rigoroso dal punto di vista fisico. Per ogni singolo raggio k emesso dal sensore, l'algoritmo fa un *raycasting* (traccia un raggio virtuale) sulla mappa m (mappa conosciuta dal robot) partendo dalla posa x_t , per calcolare la distanza z_k , ovvero la distanza a cui si trova il primo ostacolo sulla mappa.

Fatto ciò, per ogni raggio k , calcola la posizione tra posa x_t e rilevazione del sensore calcolando la distanza ideale z_k^* dove si trova l'oggetto secondo il sensore.

Infine calcola la differenza tra le due distanze:

$$\Delta = z_k - z_k^* \quad (19)$$

Tuttavia, i sensori reali sono imperfetti e il mondo è dinamico. Il modello tiene conto di **quattro tipi di errore indipendenti**, modellati matematicamente e sommati per ottenere la probabilità finale di una singola lettura z_k .

1. **Errore di Misura Gaussiano (p_{hit}):** È il rumore standard del sensore. Se l'ostacolo è a 3m, il sensore potrebbe leggere 3.02m o 2.98m. Viene modellato con una campana di Gauss centrata sulla distanza attesa z_k^* .
2. **Ostacoli Inattesi Dinamici (p_{short}):** Il raggio colpisce un oggetto che *non* è sulla mappa (es. una persona che cammina o una sedia spostata). Questi ostacoli accorciano sempre la lettura rispetto a z_k^* . Viene modellato con un decadimento Esponenziale.
3. **Fallimento del Sensore (p_{max}):** A volte la luce laser viene assorbita (es. superfici nere) o riflessa via (es. specchi/vetri). In questi casi, il sensore non riceve il segnale di ritorno e restituisce per default il suo valore massimo possibile (es. $z_{max} = 30m$). Viene modellato con un singolo picco (Delta di Dirac) sul valore massimo.
4. **Misurazioni Casuali / Rumore Fantasma (p_{rand}):** Interferenze elettromagnetiche o riflessi multipath possono generare letture totalmente casuali e prive di senso. Viene modellato con una distribuzione Uniforme piatta su tutto il range del sensore.

La probabilità finale per un singolo raggio è la somma pesata (mistura) di queste quattro distribuzioni:

$$p(z_k|x_t, m) = w_{hit}p_{hit} + w_{short}p_{short} + w_{max}p_{max} + w_{rand}p_{rand} \quad (20)$$

Nota: La somma dei pesi w deve essere pari a 1.

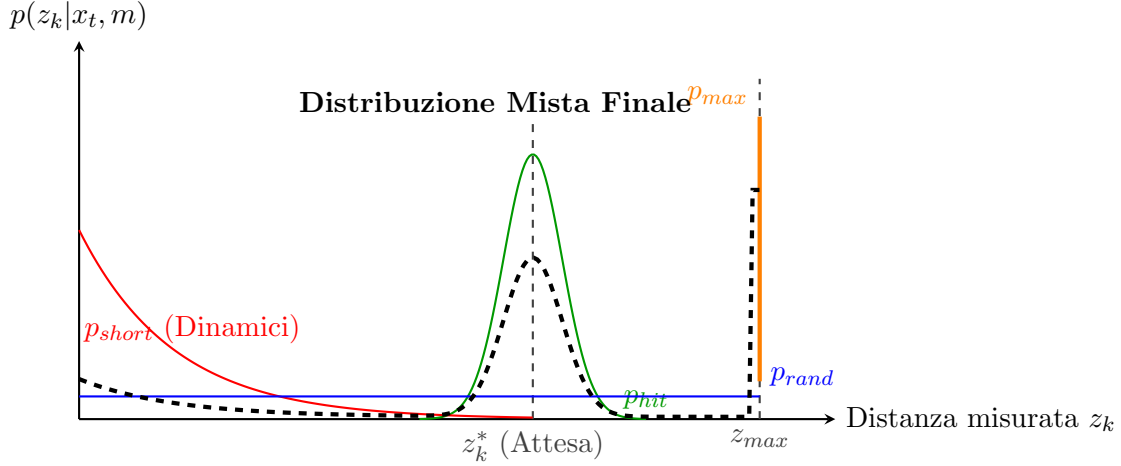


Figura 10: Le quattro componenti del Beam-based Model. La curva nera tratteggiata rappresenta la probabilità complessiva che il sensore legga una specifica distanza z_k .

Infine quindi usa il Δ per trovare sulla gaussiana appena calcolata (la quale è centrata in zero), la probabilità che quel punto preso in considerazione sia la posizione reale del robot.

Il Problema del Beam-based Model:

Nonostante sia fisicamente accurato, questo modello presenta due gravi difetti computazionali e di robustezza. Primo, richiede di simulare il *raycasting* sulla griglia per ogni particella e per ogni raggio laser ad ogni ciclo (un carico CPU enorme). Secondo, in ambienti non perfettamente lisci o con lievi difetti della mappa, il modello è eccessivamente severo, facendo crollare la probabilità a zero in presenza di piccole variazioni.

5.2 Likelihood Field Model (Scan Model)

Per ovviare ai problemi computazionali del Beam-based, i filtri moderni utilizzano il **Likelihood Field Model**. Invece di calcolare l'intera traiettoria del raggio laser, questo modello valuta esclusivamente l'utile finale: *il punto esatto (end-point) in cui il laser ha colpito qualcosa*.

L'algoritmo procede nel seguente modo:

1. **Proiezione dell'end-point:** Data la posa ipotetica del robot x_t e la distanza letta z_k , proietta geometricamente nello spazio le coordinate (x_{end}, y_{end}) in cui l'ostacolo è stato rilevato.
2. **Ricerca del vicino più prossimo:** Controlla sulla mappa globale m qual è l'ostacolo statico più vicino a quel punto (x_{end}, y_{end}) calcolando la distanza Euclidea d .
3. **Valutazione Gaussiana:** Se l'ostacolo reale è molto vicino al punto rilevato (d piccolo), la probabilità è alta. Se l'end-point cade in una zona libera della mappa lontana da qualsiasi muro (d grande), la probabilità scende. Il tutto è modellato con una Gaussiana a media nulla e con varianza fornita dal produttore del sensore.

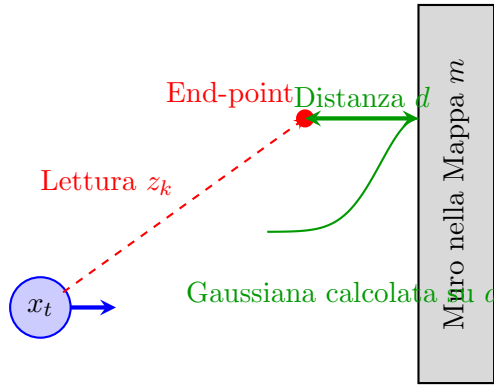


Figura 11: Il Likelihood Field Model valuta solo l'end-point del raggio. Più l'end-point è distante da un ostacolo tracciato sulla mappa (distanza d), minore sarà la probabilità assegnata a quella posa.

Per ottimizzare ulteriormente i tempi di calcolo, la mappa m non viene interrogata al momento dell'esecuzione. Viene invece pre-calcolata una mappa speciale in memoria (una *Distance Transform*), in cui ogni cella vuota della mappa contiene già il valore pre-calcolato della distanza dall'ostacolo più vicino. In questo modo, l'aggiornamento del Likelihood Field diventa un banale e istantaneo accesso in memoria matrice $O(1)$.